堆排序

二叉堆是一种特殊的堆, 二叉堆是完全二叉树或者近似完全二叉树.

堆排序是利用二叉堆的特性, 对根节点(最大或最小)进行循环提取, 从而达到排序目的(堆排序本质上是一种选择排序), 时间复杂度为 `O(logn)` 堆排序是不稳定排序

	插入	删除	寻找最大/最小值
无序数组	1	n	n
无序链表	1	n	n
有序数组	n	1	1
有序链表	n	1	1
二叉堆	logn	logn	1
二叉树	logn(平均)	logn(平均)	logn(平均)

二叉树

是指树中节点的度不大于2的有序树，它是一种最简单且最重要的树。

满二叉树

除最后一层无任何子节点外，每一层上的所有结点都有两个子结点的二叉树。

从图形形态上看，满二叉树外观上是一个三角形。

如果一个二叉树的层数为K，且结点总数是(2^k) -1 ，则它就是满二叉树。

完美二叉树

除去叶子节点, 所有节点的度都是 2。也就是说，所有的节点的度只能是0或2。

完全二叉树

一棵深度为k的有n个结点的二叉树，对树中的结点按从上至下、从左到右的顺序进行编号，如果编号为i（1≤i≤n）的结点与满二叉树中编号为i的结点在二叉树中的位置相同，则这棵二叉树称为完全二叉树。

- 除了最后一层外, 所有层都完美填充
- 最后一层所有叶子节点靠左对齐

堆

堆是一棵完全二叉树。核心思想是，先建堆，后调整。

堆总是满足下列性质：

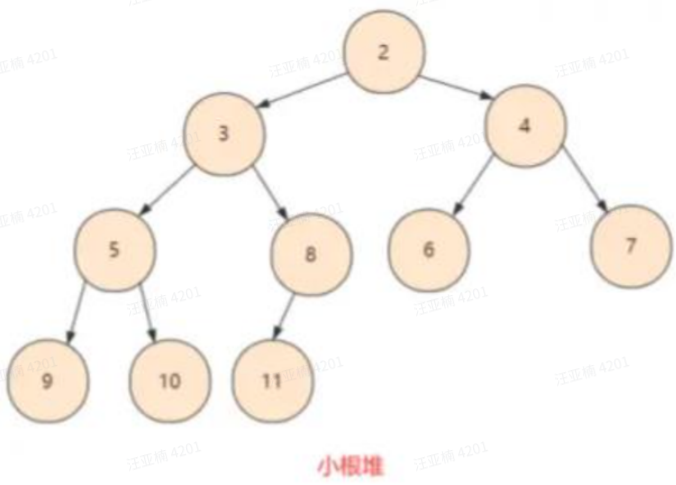
- 堆总是一棵完全二叉树；
- 堆中某个节点的值总是不大于或不小于其父节点的值；

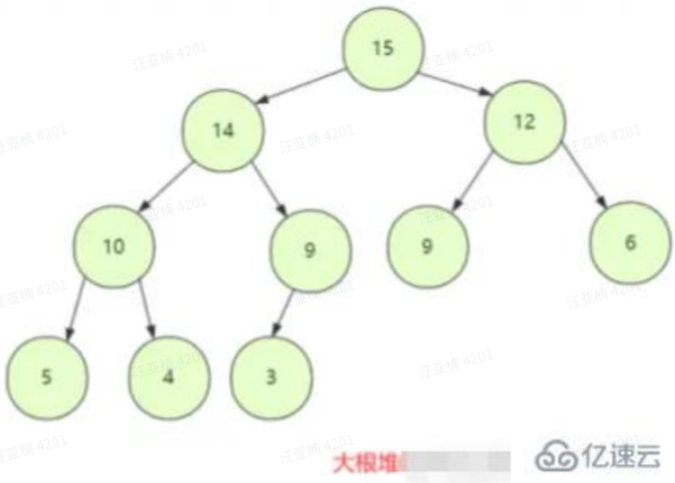
最大堆（大顶堆）

- 如果所有节点 大于 孩子节点值，那么这个堆叫做 **最大堆**，堆的最大值在根节点。

最小堆（小顶堆）

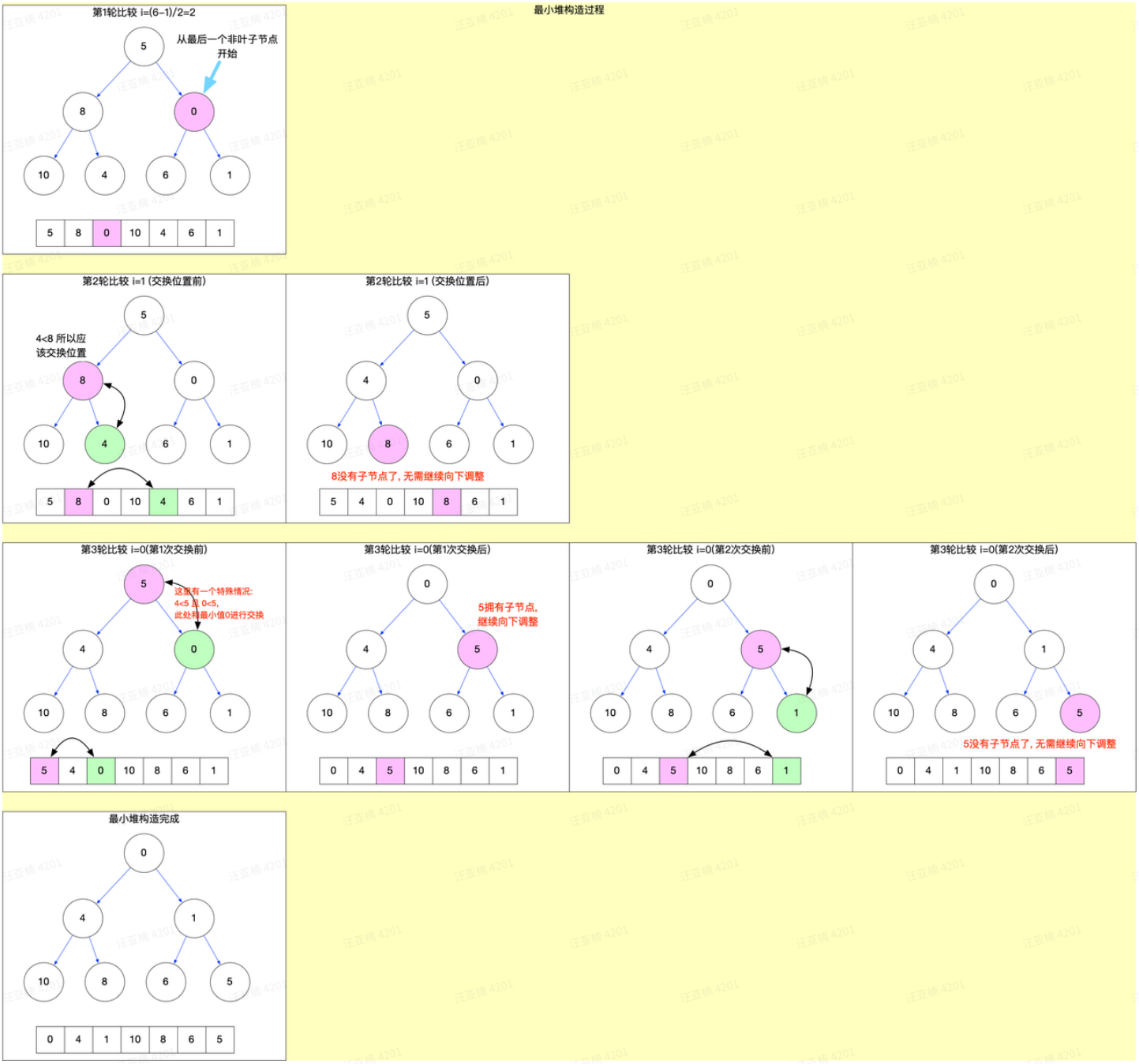
- 如果所有节点 小于 孩子节点值，那么这个堆叫做 **最小堆**，堆的最小值在根节点。



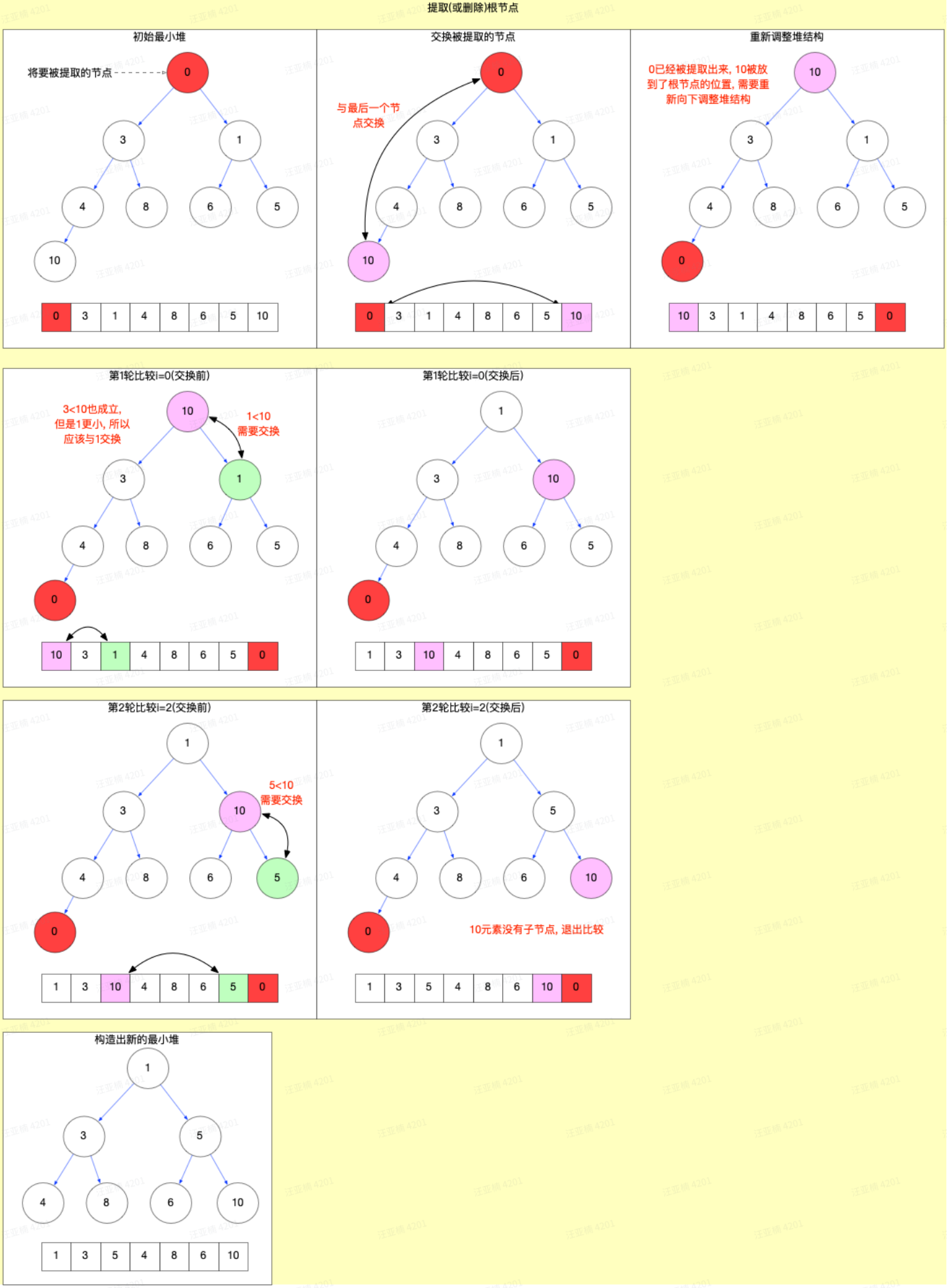


假设现在有一个乱序数组, [5,8,0,10,4,6,1], 现在将其构造成一个最小堆

1. 构造二叉堆
    - 需要从最后一个非叶子节点开始, 向下调整堆结构



1. 插入节点，重新向上调整堆(`sift-up`)
   - 将新元素插入到数组末尾之后，要重新调整数组结构，保证数组任然是最小(或最大)堆.
1. 提取或删除根节点(顶端节点)，重新向下调整堆(`sift-down`)
   - 对于最大堆，提取的是最大值. 对于最小堆，提取的是最小值.
   - 顶点被提取之后，要重新调整数组结构，保证数组任然是最小(或最大)堆.


提取(或删除)根节点
初始最小堆
将要被提取的节点
交换被提取的节点
与最后一个节点交换
重新调整堆结构
0已经被提取出来，10被放到了根节点的位置，需要重新向下调整堆结构
第1轮比较i=0(交换前)
3<10也成立，但是1更小，所以应该与1交换
1<10 需要交换
第1轮比较i=0(交换后)
第2轮比较i=2(交换前)
5<10 需要交换
第2轮比较i=2(交换后)
10元素没有子节点，退出比较
构造出新的最小堆

1. 排序过程

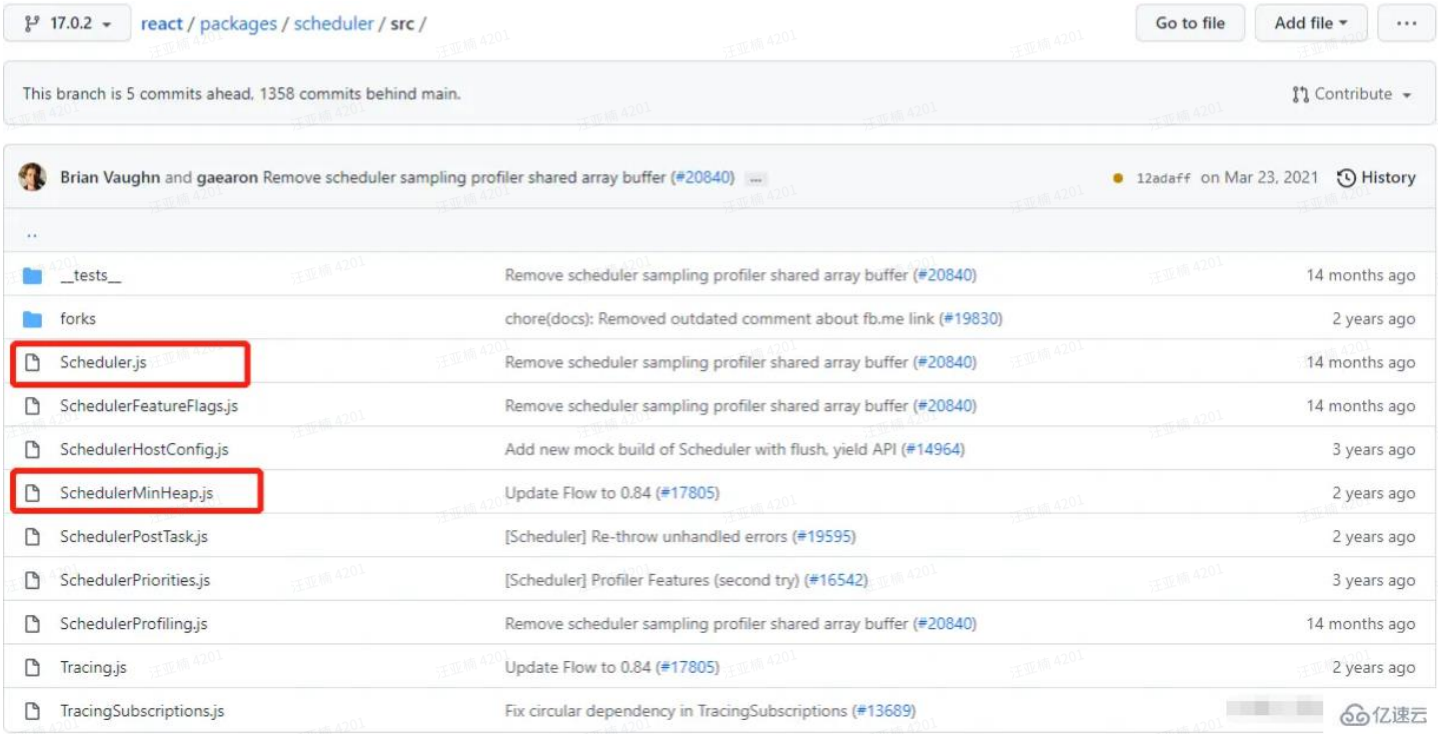
利用二叉堆的特性, 排序就是循环提取根节点的过程. 循环执行步骤 3, 直到将所有的节点都提取完成, 被提取的节点构成的数组就是一个有序数组.

注意:

- 如需升序排序, 应该构造最大堆. 因为最大的元素最先被提取出来, 被放置到了数组的最后, 最终数组中最后一个元素为最大元素.
- 如需降序排序, 应该构造最小堆. 因为最小的元素最先被提取出来, 被放置到了数组的最后, 最终数组中最后一个元素为最小元素.
- 堆排序是一种不稳定排序(对于相同大小的元素, 在排序之后有可能和排序前的先后次序被打乱).

React源码部分

React源码中的目录packages/scheduler，就是React的任务调度模块相关的代码。



React 当中的使用场景

对于二叉堆的应用是在 `scheduler` 包中, 有 2 个数组 `taskQueue` 和 `timerQueue`, 它们都是以 最小堆 的形式进行存储, 这样就能保证以 `O(1)` 的时间复杂度, 取到数组顶端的对象(优先级最高的 task).